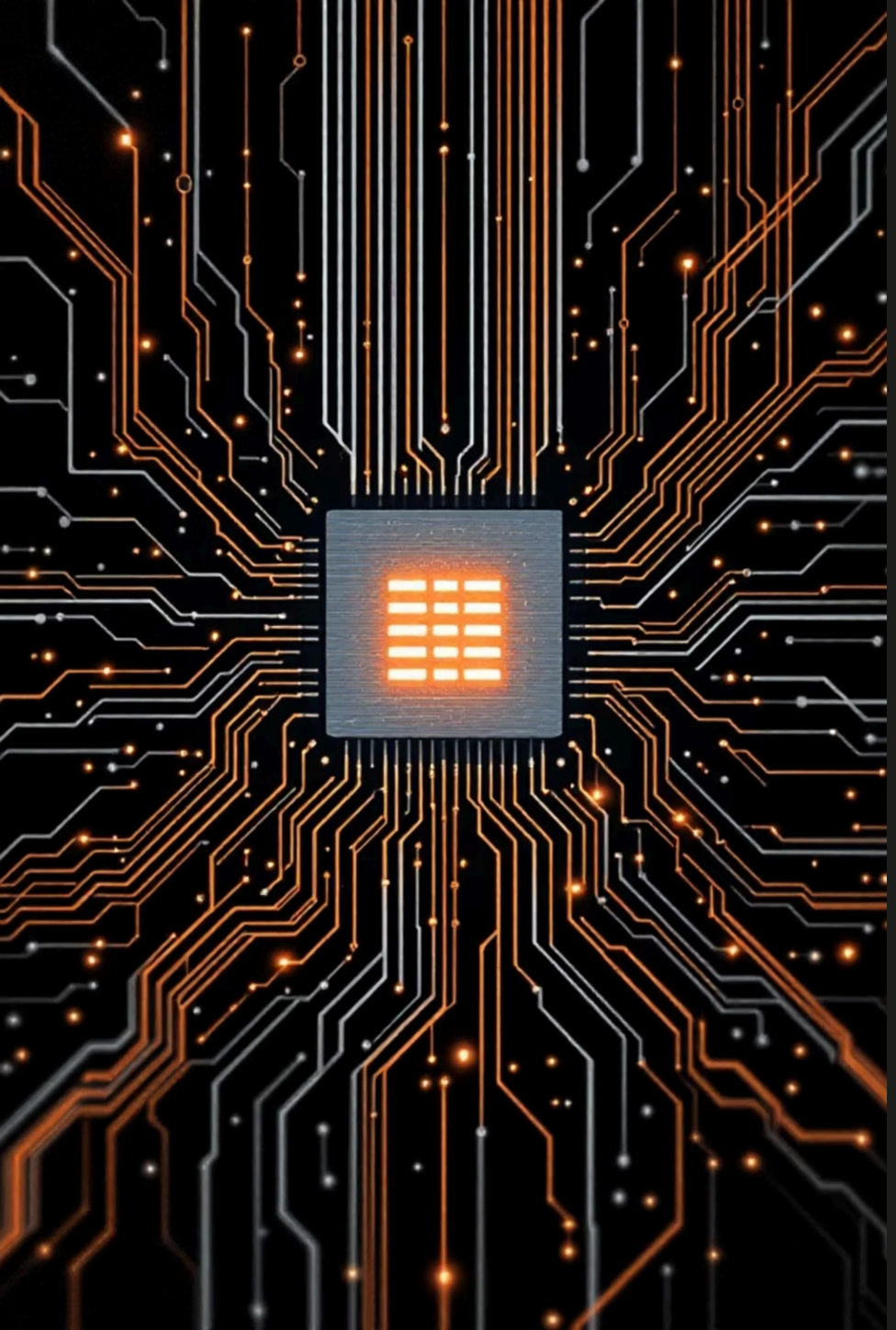


Rust: Segurança na Programação

Modelo de Ownership

Nuno Neto - 2026

CONNECT5
EDIH European Digital Innovation Hubs Network
U.PORTO FACULDADE DE CIÊNCIAS UNIVERSIDADE DO PORTO
REPÚBLICA PORTUGUESA
Financiado pela União Europeia NextGenerationEU
PRR Plano de Recuperação e Resiliência

O que é o Modelo de Ownership?

O Rust usa verificações **estáticas** (*compile-time*) e **dinâmicas** (*runtime*) para evitar comportamentos indefinidos, maximizando sempre as verificações estáticas.

Ownership é o mecanismo estático do Rust para proteger a gestão de memória.

Memória não inicializada

Impede leitura de dados inválidos

Use after free

Bloqueia acesso após destruição

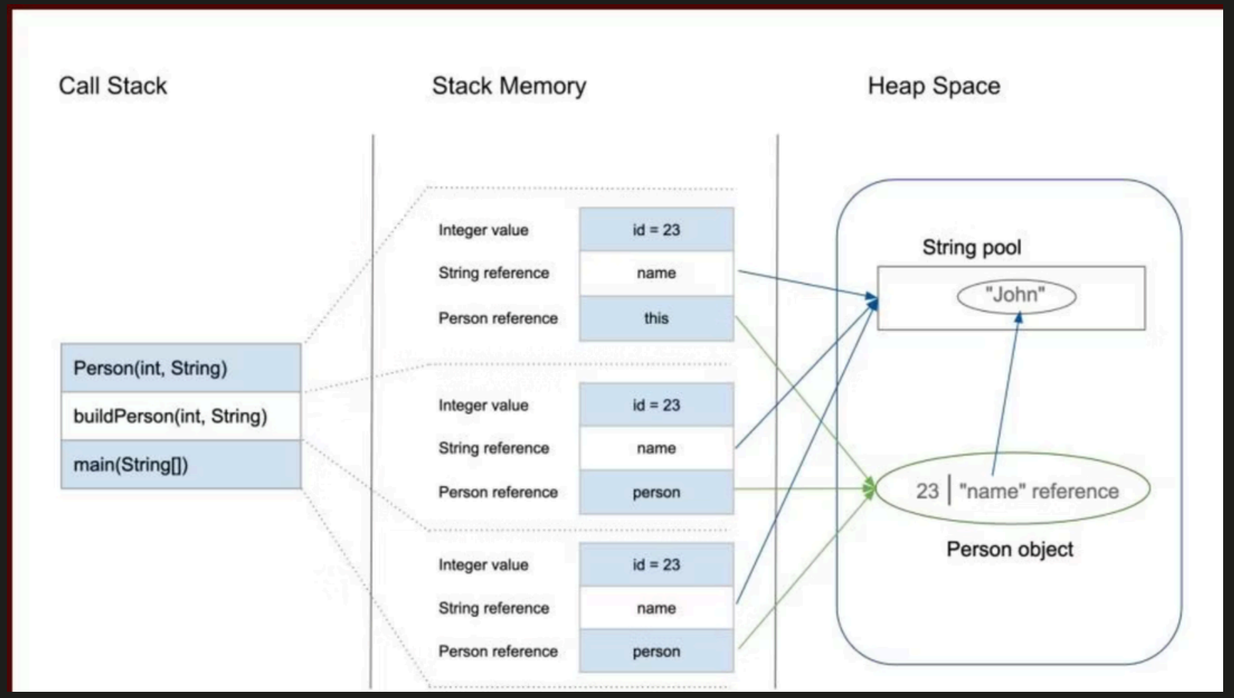
Double free

Evita libertar memória duas vezes

Memory leaks

Garante limpeza automática

Stack vs. Heap



Stack

- LIFO, tamanho fixo em *compile time*
- Extremamente rápida
- Gestão simples, ligada às *scopes*

Heap

- Tamanhos flexíveis e dinâmicos
- Mais lenta pois requer interação com o OS
- Gerida por GC, manualmente ou por Ownership

As 3 Regras de Ownership

Um dono

Cada valor em Rust tem **um único dono**.

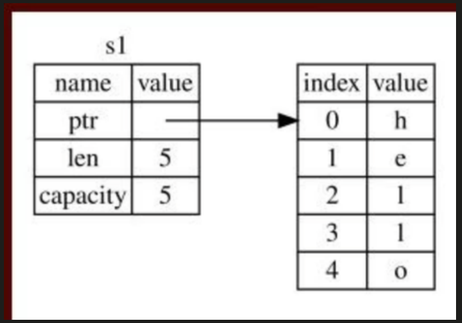
Exclusividade

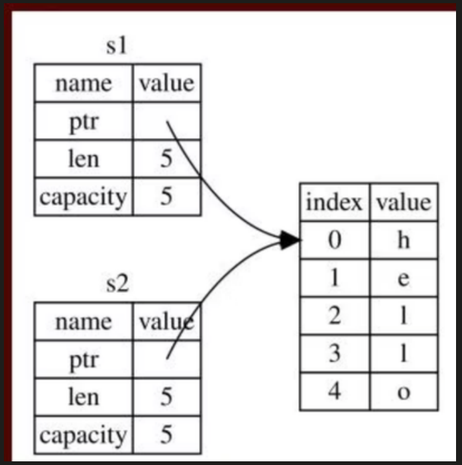
Só pode haver **um dono em qualquer momento**.

Destruição automática

Quando a **scope** do dono termina, todas as variáveis são destruídas.

Movimentação de Dados

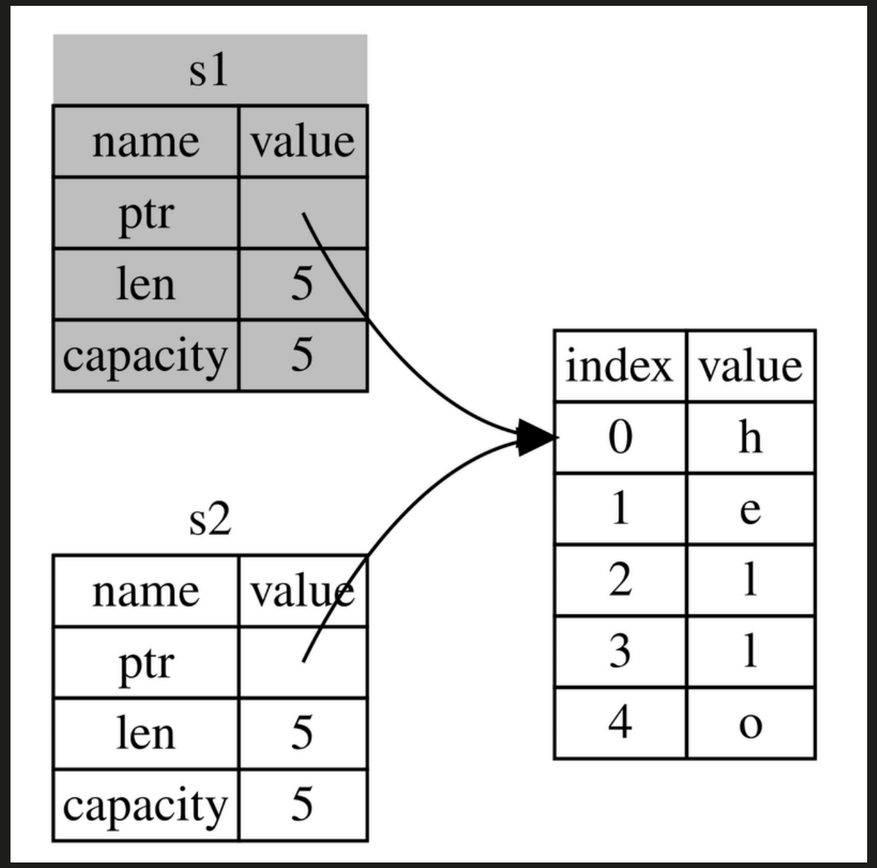




Ao atribuir let s2 = s1, uma cópia *shallow* partilharia o mesmo ponteiro na heap — causando **double free** ao sair da scope.

A cópia profunda (*deep copy*) *por default* foi rejeitada por ser custosa e deve ser feita explicitamente.

A solução do Rust: o valor é **movido**. A ownership passa para s2 e s1 é invalidado.



Tipos Copy e Clone

Trait Copy

Tipos de tamanho fixo na **stack** (ex: i32). A cópia é automática e eficiente.
Automaticamente derivável para tipos que são composições de outros tipos copy.

Tipos Copy não podem implementar Drop.

Trait Clone

Permite *deep copy* explícita via .clone(). Cria um novo objeto independente, sem eliminar o original. Pode ser implementada para qualquer tipo.
Representa a *deep copy* e tem de ser chamada explicitamente.

Ownership e Funções

```
fn main() {
    let s = String::from("teste");
    adota_objeto(s); // s é movido
    // s já não é válido aqui!

    let x: i32 = 5;
    copia_objeto(x); // x é Copy
    // x continua válido aqui
}

fn adota_objeto(s: String) {
    println!("{}", s);
} // drop de s é chamado, memória devolvida

fn copia_objeto(n: i32) {
    println!("{}", n);
} // n sai da stack, sem drop
```

Ao chamar uma função, a ownership dos argumentos é **transferida** para a scope da função — tal como numa atribuição.

Retornar valores segue o mesmo padrão: a ownership passa para o *caller*. Valores não retornados são destruídos.

> Para manter a posse de um argumento, é necessário **retorná-lo** explicitamente ou incorrer numa cópia completa.

Desvantagem do Modelo de Ownership

Com estas regras, torna-se difícil passar argumentos a funções sem perder a sua posse.

Retornar argumentos

Devolver os valores cujo controlo não queremos perder — verboso e trabalhoso.

Cópia completa

Clonar o objeto antes de o passar — pode ter custo elevado.

Solução futura

O Rust resolve isto com o conceito de **referências e borrowing** — próximo tema.

String Literals vs. String Objects

String Literal
&str

- Tamanho conhecido em *compile time*
- Imutável, armazenada na **Stack**
- Rápida e eficiente de copiar

String Object
String

- Tamanho dinâmico, mutável
- Armazenada na **Heap**
- Requer alocação e devolução ao SO

No Rust, a memória é devolvida automaticamente quando o dono (o objeto String) é destruído — sem GC nem gestão manual.

Esta dealocação chama uma função especial: **drop**. Este padrão é também conhecido como **RAII** (*Resource Acquisition Is Initialization*).